

Adaptive Timing Control for Throughput–Latency Trade-off in nFAPI Split

Ming-Hong HSU*, Ray-Guang Cheng*, and Co-author 1[†]

*Dept. of Electronic and Computer Engineering, National Taiwan University of Science and Technology, Taiwan

[†]Affiliation 2

Email: crg@mail.ntust.edu.tw

Abstract—In the Open Radio Access Network (O-RAN) architecture, functional split Option 6 (standardized as nFAPI) provides deployment flexibility over general packet-switched networks by decoupling the MAC and PHY layers, avoiding the very high costs of Time-Sensitive Networking (TSN). However, migrating the MAC layer to a Virtualized Network Function (VNF) in a cloud environment introduces non-deterministic transport and processing latency. Existing open-source implementations rely on static slot-ahead timing configurations, which fail to adapt to unpredictable network jitter, leading to severe scheduling failures and throughput degradation. To address this challenge, this paper proposes an adaptive timing control framework for non-ideal fronthaul environments. The proposed method employs an Exponentially Weighted Moving Average (EWMA) to dynamically estimate network delay and actively adjust the scheduling slot-ahead, ensuring control messages arrive within the strict timing window of the Physical Network Function (PNF). Experimental validations on an OpenAirInterface (OAI) testbed integrated with a commercial O-RAN Radio Unit (O-RU) demonstrate that the proposed adaptive mechanism effectively eliminates synchronization failures, maintains stable Hybrid Automatic Repeat Request (HARQ) round-trip times, and ensures stable system throughput under severe network congestion.

Index Terms—nFAPI, O-RAN, Functional Split, Adaptive Control.

I. INTRODUCTION

A. Background and Motivation

To address the diverse demands of 5G networks, the 3rd Generation Partnership Project (3GPP) and the O-RAN Alliance have standardized the disaggregation of traditional base stations into Open Central Units (O-CUs), Open Distributed Units (O-DUs), and Open Radio Units (O-RUs) [1]. Among the proposed architectures, three functional split configurations have emerged as mainstream: Option 2 (O-CU/O-DU), Option 7.2 (O-DU/O-RU), and Option 6 (O-DU High/Low) [2].

These splits present distinct synchronization requirements and management strategies. Option 2 relies on sequence numbers and deep buffering over general packet-switched networks, making its timing constraints relatively relaxed. Option 7.2 [3], operating within the physical layer, demands strict phase/symbol-level synchronization enforced by hardware Precision Time Protocol (PTP) over Time-Sensitive Networking (TSN). Occupying a unique intermediate position is Option 6 (standardized as nFAPI by the Small Cell Forum (SCF) [4]), which separates the MAC and PHY layers. Option 6 requires frame/slot-level synchronization and focuses on delay

management, offering the flexibility to operate over general packet-switched networks without the very high costs of TSN.

The rationale for deploying the MAC layer (O-DU High) in a cloud environment via Option 6 is primarily driven by resource pooling, as Layer 2 processing complexity scales dynamically with user equipment (UE) density [5]. However, migrating the MAC layer to a Virtualized Network Function (VNF) introduces internal processing latency that compresses the available margin for fronthaul transmission delay. Unlike Option 7.2, which rigidly drops out-of-window packets, Option 6 must adaptively manage delays to ensure slot-level synchronization over non-deterministic Ethernet transport. Existing open-source implementations typically employ static slots ahead configurations, which fail to accommodate dynamic network jitter and server processing variations, leading to frequent scheduling failures (e.g., missing slot packet).

This highlights a fundamental gap in current disaggregated RAN designs: the lack of adaptive timing control mechanisms for intermediate-layer splits operating under non-ideal transport environments.

B. Related Works

Recent advancements in Open RAN (O-RAN) architectures have driven significant research into optimizing split-architecture performance and managing network latency. For instance, the X5G testbed proposed by Villa *et al.* [6] demonstrates a state-of-the-art approach by deploying a hardware-accelerated private 5G O-RAN network. By offloading Physical (PHY) layer processing to dedicated NVIDIA GPUs and Mellanox SmartNICs, X5G successfully achieves commercial-grade peak throughputs. However, this high performance relies heavily on expensive, specialized hardware and demands strict, ideal network conditions equipped with hardware-level Precision Time Protocol (PTP) synchronization. Such stringent requirements significantly limit its applicability and deployment flexibility in general packet-switched networks, where unpredictable latency and jitter are prevalent.

Beyond hardware acceleration, software-driven delay management strategies have also been explored from various perspectives. At the intra-node computational level, Lee *et al.* [7] proposed an adaptive Layer 1 transmission strategy that opportunistically advances transmissions when software processing finishes early. From a network orchestration perspective, Adamuz-Hinojosa *et al.* [8] introduced ORANUS,

employing Stochastic Network Calculus (SNC) to provide mathematical latency bounds for ultra-reliable low-latency communications (uRLLC). Moreover, Lv *et al.* [9] emphasized User Equipment (UE) QoS support by integrating MAC resource allocation with high-level network orchestration. While these frameworks [7]–[9] provide robust solutions for processing variance, capacity bounds, and QoS scheduling, they do not directly address the critical synchronization failures caused by unpredictable fronthaul network jitter in intermediate-layer splits.

To address the limitations of high hardware costs and strict synchronization requirements, this paper proposes a cost-effective, software-based solution tailored for non-ideal fronthaul environments. Specifically, we focus on the fact that the Small Cell Forum (SCF) 225 nFAPI specification [4] defines the standard *Timing Info* message and interface-level delay management fields, yet existing open-source implementations only support static parameter initialization. Our work introduces an adaptive slot-ahead timing control mechanism based on Exponentially Weighted Moving Average (EWMA) delay management to complete this standardized dynamic feedback loop. This approach dynamically tracks and filters network jitter using general-purpose Commercial Off-the-Shelf (COTS) servers. Experimental results demonstrate that our proposed method ensures stable Hybrid Automatic Repeat Request (HARQ) Round-Trip Times (RTT) and stable system throughput without the need for high-end specialized equipment or strict hardware synchronization, thereby offering a more flexible and economically viable alternative for O-RAN deployments.

Specifically, compared to Split 7.2 implementations like X5G [6] that require hardware-level PTP synchronization and GPU/SmartNIC offloads, or FAPI-based Split 6 designs [7] restricted to intra-node shared memory and process-level adjustments, our proposed framework operates at the slot/TTI level on standard COTS servers, actively managing cross-node network jitter and delay over standard packet-switched Ethernet without hardware acceleration dependencies.

C. Research Scope and Contributions

In this paper, we focus on the nFAPI-based Option 6 split as a representative case of timing-sensitive yet transport-flexible RAN disaggregation. While Option 6 offers significant advantages in terms of reduced fronthaul bandwidth and deployment flexibility, its performance is highly sensitive to non-deterministic latency introduced by transport networks and server processing.

We identify that the root cause of performance degradation in such systems lies in the inability of the MAC scheduler to adapt its transmission timing to dynamic end-to-end latency variations. Existing open-source implementations, such as OpenAirInterface (OAI), typically employ static slots ahead configurations, which fail to accommodate fluctuating network conditions, leading to frequent synchronization failures and throughput degradation.

To address this issue, we propose an adaptive timing control framework that dynamically adjusts scheduling lead time based on real-time latency estimation. The proposed approach consists of two main components: Node Sync, which aligns frame and slot boundaries between the VNF and PNF, and Delay Management, which applies an Exponentially Weighted Moving Average (EWMA) to smooth the timing information and dynamically adjust the slots ahead to ensure that scheduling messages arrive within the valid timing window.

The main contributions of this paper are summarized as follows:

- **Systematic Analysis of Timing Uncertainty in Split 6:** We characterize the impact of non-deterministic latency, including network jitter and processing delay, on synchronization stability in disaggregated RAN systems.
- **Adaptive Slots Ahead mechanism:** We propose a dynamic timing adjustment mechanism that continuously aligns scheduling decisions with varying transport delays, effectively mitigating timing violations.
- **Experimental Validation on OAI Platform:** We implement the proposed framework in a real-world testbed and demonstrate that it eliminates missing slot packet errors and significantly improves throughput stability under non-ideal fronthaul conditions.
- **Open-Source nFAPI Platform Contribution to Upstream OAI:** We submitted our adaptive timing control and nFAPI modifications as a merge request (MR) to the official OpenAirInterface (OAI) repository. This contribution was merged into the official codebase. It provides a standard open-source nFAPI platform that follows the Small Cell Forum (SCF) specification for future development and testing.

II. SYSTEM MODEL

Before diving into the system details, this study introduces the adopted system framework. To integrate the Network Functional Application Platform Interface (nFAPI) with mainstream commercial equipment, the proposed system adopts a stable two-stage split architecture. The network first connects via the Split 6 nFAPI interface between the network functions, and then translates to the standard O-RAN Split 7.2x interface to connect to a commercial Pegatron O-RU. Figure 1 illustrates this proposed system architecture and its delay management mechanism.

A. End-to-End Architecture

The system establishes a complete 5G end-to-end communication link, extending from the core to the user equipment:

- **Core Network (CN):** The central element responsible for routing, session management, and authenticating user data.
- **O-RAN Central Unit (O-CU):** A logical node that handles higher-layer protocol stacks, such as the Radio Resource Control (RRC) and Packet Data Convergence Protocol (PDCP). It connects with distributed units via the F1 interface.

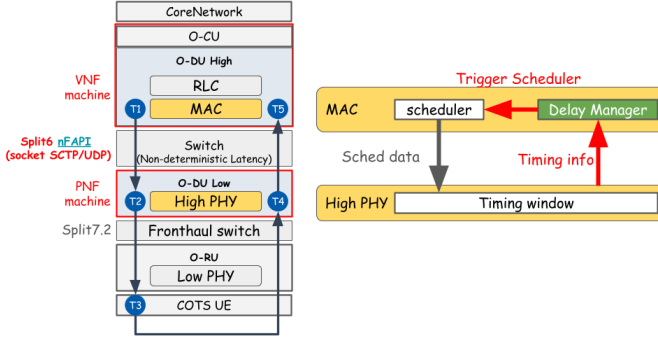


Fig. 1. System architecture and delay management mechanism.

- **O-RAN Distributed Unit (O-DU):** To optimize computational performance and deployment flexibility, the system separates the O-DU into an O-DU High and an O-DU Low. The O-DU High is executed as a Virtualized Network Function (VNF) on a virtualized machine, whereas the O-DU Low is deployed as a Physical Network Function (PNF) on dedicated hardware.
- **Switch:** A commercial network switch is located between the VNF and PNF. This mid-layer connection utilizes the nFAPI protocol. However, traversing this packet-switched network inevitably generates non-deterministic latency.
- **O-RAN Radio Unit (O-RU):** A specialized hardware unit responsible for low-level physical layer (Low PHY) processing, digital beamforming, and Radio Frequency (RF) transmission.
- **Commercial Off-The-Shelf User Equipment (COTS UE):** Standard commercial smartphones or modems that act as the terminal receivers in the network.

B. O-DU High: MAC, Scheduler, and Dual-Loop Timing Core

Within the O-DU High virtualized network function, the timing control is organized as a Dual-Loop Control System. The core components handling this timing core include the Scheduler, the Node Sync Module (Inner-Loop), and the nFAPI Delay Manager (Outer-Loop):

- **Scheduler:** Critical for allocating the available radio resources (such as time-frequency resource blocks) and determining the exact priority and timing of downlink/uplink data transmissions.
- **Node Sync Module (Inner-Loop):** Responsible for software-level clock synchronization. This module periodically exchanges NTP-style timestamps ($t_1 \sim t_4$) with the PNF via standard *DL/UL_node_sync* messages. It calculates the instantaneous clock offset and applies slot-level (S_{adj}) and microsecond-level (μS_{adj}) adjustments to align the VNF's local clock to the PNF baseline, eliminating long-term hardware clock drift.
- **nFAPI Delay Manager (Outer-Loop):** Operates on top of the stabilized clock phase established by the inner loop. The delay manager utilizes the arrival offset (Δt_{arrive})

feedback from the PNF to dynamically estimate transport network jitter. It then actively adjusts the slots ahead (S_{ahead}) trigger offset of the scheduler. This outer-loop control compensates for the non-deterministic packet transport delays and VNF processing delays without inducing synchronization loss.

C. O-DU Low: High PHY and Timing Window

The O-DU Low manages the higher-level baseband processing of the physical layer (High PHY). At this level, the most critical synchronization mechanism is the nFAPI PNF Timing Window:

- **Timing Window:** A predefined, strict time duration. When the scheduling message (specifically the *DL_TTI.request*) sent by the O-DU High propagates across the network, it must accurately arrive and fall exactly within this Window. Otherwise, the High PHY will reject the message and fail to process the slot data.
- **Timing Info (Δt_{arrive}):** The PNF continuously monitors the exact arrival offset of control messages relative to the transmission deadline. This offset, denoted as Δt_{arrive} , represents the remaining time margin within the timing window. The PNF periodically sends this information back to the Delay Manager in the MAC layer. This establishes a closed-loop feedback control system, compensating for the unpredictable transmission latency variations between the VNF and PNF machines.

D. Key Validation Metrics

To evaluate the stability, synchronization accuracy, and real-time performance of the proposed algorithm under non-ideal environments, this study defines the following Key Validation Metrics:

- **MAC Layer HARQ RTT ($T_5 - T_1$):** Measures the Hybrid Automatic Repeat Request (HARQ) loop delay specifically at the MAC layer. This metric accurately reflects the feedback efficiency and operational health of the link layer.
- **VNF-PNF DL Latency ($T_2 - T_1$):** Measures the one-way downlink latency accumulated from the virtualized machine (VNF) to the physical machine (PNF). This metric is used to directly quantify and analyze the delay jitter impact injected by the intermediate Ethernet switch.

Note that RTT denotes the Round-Trip Time. To validate the split-architecture stability under non-ideal fronthaul conditions, network latency simulation is conducted at the uplink ($T_5 - T_4$) and downlink ($T_2 - T_1$) segments via Linux Traffic Control (TC).

To validate the proposed performance optimization within a realistic network environment, the system design leverages a commercial-grade deployment framework:

- **O-RAN Split 7.2x Interoperability:** While the Small Cell Forum (SCF) has standardized the nFAPI for the MAC-PHY split (Option 6) [10], commercial-grade Open Radio Units (O-RUs) natively support the O-RAN Split

Option 7.2x. To achieve seamless integration with open-source stacks like OpenAirInterface [11], our system bridges these interfaces.

- **Hybrid System Architecture for E2E Validation:** To validate the nFAPI performance over a fully commercial ecosystem, this paper adopts a hybrid architecture (Split Option 6 + Split Option 7.2). This setup establishes a translation/interworking layer that enables the nFAPI-driven MAC/High-PHY to successfully interact with a commercial Pegatron O-RU, facilitating end-to-end verification using Commercial Off-The-Shelf (COTS) UEs.

III. PROPOSED METHOD

This section details the proposed dynamic timing adjustment algorithm. To resolve the unstable latency during packet transmission between the PNF and VNF, we propose a Dual-Loop Timing Architecture consisting of software-level clock synchronization and slot-level adaptive delay management.

A. Dual-Loop Timing Architecture

The timing control of this system is organized as a closed-loop Dual-Loop Control System, ensuring that the VNF-side scheduling can dynamically track and compensate for both clock drift and slot-level transport latency variation. The system establishes two primary loops:

The two control loops cooperate as follows:

- 1) **Inner-Loop (Node Sync):** Maintains phase and slot boundary alignment between the VNF and PNF. It periodically exchanges timestamps to calculate the absolute phase offset, correcting for hardware clock drift.
- 2) **Outer-Loop (Delay Management):** Tracks and estimates packet transport delays and jitter at the slot level, dynamically adjusting the slots ahead (S_{ahead}) to guarantee that control messages fall within the PNF's strict timing window.

B. Inner-Loop: Node Sync

To perform software-level clock synchronization, the system uses a Network Time Protocol (NTP) style timestamp exchange. When the VNF receives the UL_node_sync message, it calculates the instantaneous timing offset ($offset$) based on four timestamps (t_1 to t_4). To avoid instantaneous spikes and prevent the system from reacting overly sensitively to raw input data, a smoothing preprocessing step is necessary. However, traditional moving averages incur high computational costs to record and maintain historical data within a sliding window. Therefore, the algorithm uses an Exponentially Weighted Moving Average (EWMA) model to process the $offset$ and its variation ($error$). This design exponentially decays the weight of historical data while applying only a specific weight to new data, ensuring the system is responsive yet stable. Specifically, it generates an average offset (μ_{offset}) and a mean absolute deviation (dev_{offset}). The smoothing factors α and β are used for these updates. Drawing inspiration from the TCP Congestion Avoidance and Control mechanisms specified in RFC 6298 [12], [13], we set $\alpha = 1/8$

for the mean offset and $\beta = 1/4$ for the jitter variation. This configuration, proven effective in TCP over many years, makes jitter detection more sensitive. Furthermore, to improve execution efficiency, these factors are designed as negative powers of 2 ($\alpha = 2^{-3}$, $\beta = 2^{-2}$), allowing the computation logic to be implemented using highly efficient bitwise shift operations. A dynamic synchronization threshold ($Threshold$) is then derived directly from the estimated jitter (dev_{offset}). If the absolute $offset$ exceeds this $Threshold$, the system breaks the $offset$ down into a slot adjustment (S_{adj}) and a microsecond-level adjustment ($\mu_{s_{adj}}$) based on the slot duration ($slot_duration$). These parameters are subsequently consumed by the VNF Timing Actuator to precisely align the local clock. The process is summarized in Algorithm 1.

Algorithm 1 Node Sync

Require: t_1, t_2, t_3 (Timestamps from PNF UL_node_sync), t_4 (VNF local receive timestamp)

Ensure: S_{adj} (Slot adjustment), $\mu_{s_{adj}}$ (Microsecond adjustment)

Notation:

$offset$ (Instantaneous timing offset), μ_{offset} (EWMA mean offset)

dev_{offset} (EWMA mean absolute deviation / Jitter)

$Threshold$ (Dynamic synchronization threshold)

α, β (Smoothing factors, where $\alpha = 1/8, \beta = 1/4$)

$slot_duration$ (Slot duration in μs)

Phase I: Timing Offset Calculation

1: $d_1 \leftarrow t_2 - t_1$

2: $d_2 \leftarrow t_4 - t_3$

Phase II: Offset and Dynamic Jitter Estimation

3: $offset \leftarrow (d_1 - d_2)/2$

4: $error \leftarrow offset - \mu_{offset}$

5: $\mu_{offset} \leftarrow \mu_{offset} + \alpha \cdot error$

6: $dev_{offset} \leftarrow dev_{offset} + \beta(|error| - dev_{offset})$

7: $Threshold \leftarrow dev_{offset}$

Phase III: Clock Synchronization

8: **if** $|offset| > Threshold$ **then**

9: $S_{adj} \leftarrow \lfloor offset / slot_duration \rfloor$

10: $\mu_{s_{adj}} \leftarrow offset \pmod{slot_duration}$

11: Apply S_{adj} and $\mu_{s_{adj}}$ to adjust local clock

12: **else**

13: State $\leftarrow$ **Synchronized**

14: $S_{adj} \leftarrow 0, \mu_{s_{adj}} \leftarrow 0$

15: **end if**

16: **return** $S_{adj}, \mu_{s_{adj}}$

C. Outer-Loop: Delay Management

To counteract changing network latency and maintain scheduling stability, the outer loop implements a dynamic delay management strategy. This strategy operates as an Asymmetric Feedback Controller based on a Risk-Aware Predictive Model, incorporating standard telecommunication jitter measurements to calculate and manage risk debt.

1) *Standardized Interarrival Jitter Calculation*: Rather than using an ad-hoc smoothing model, the outer loop strictly implements the standardized interarrival jitter calculation specified in IETF RFC 3550 (RTP) Section 6.4.1. Let R_i be the arrival timestamp of packet i at the physical layer (PNF), and S_i be the transmit timestamp extracted from the nFAPI P7 header of packet i . The packet transit time difference D_i between consecutive packets is defined as:

$$D_i = (R_i - R_{i-1}) - (S_i - S_{i-1}) \quad (1)$$

The smoothed interarrival jitter J_i is updated recursively using the Exponentially Weighted Moving Average (EWMA) model:

$$J_i = J_{i-1} + \frac{|D_i| - J_{i-1}}{16} \quad (2)$$

where the divisor 16 serves as the smoothing factor, optimizing the trade-off between noise reduction and responsiveness under carrier-grade network conditions. This standardized jitter metric is periodically reported by the PNF within the nFAPI Timing Info message.

2) *Asymmetric Predictive Feedback Control*: Upon receiving the worst-case late arrival offset (Δt_{arrive}) and the standardized jitter J_i reported by the PNF, the VNF Delay Manager evaluates the predicted tail risk ($Risk_{pred}$) for the next scheduling cycle:

$$Risk_{pred} = \max(\Delta t_{arrive}, \mu_{delay}) + J_i \quad (3)$$

where μ_{delay} is the EWMA mean of the late arrival offset. To prevent rapid oscillations, we introduce a history-weighted risk debt ($Debt_{risk}$), which represents a decaying memory of past timing instability:

$$Debt_{risk} \leftarrow Debt_{risk} + \alpha \cdot (\max(0, Risk_{pred}) - Debt_{risk}) \quad (4)$$

The update of the slots ahead S_{ahead} is then governed by the Fast-Attack / Slow-Release (FASR) asymmetric control law. Under FASR, the controller reacts instantly to positive risk to preserve connection reliability, while slowly relaxing the timing margin only when the system has remained stable for a sustained duration:

- **Fast-Attack (Preserve Throughput)**: If a timing violation is detected ($\Delta t_{arrive} > 0$) or the tail risk is positive ($Risk_{pred} > 0$), the controller immediately increments the slots ahead parameter to prevent scheduling timeouts:

$$S_{next} = \min(S_{curr} + 1, S_{max}) \quad (5)$$

and resets the stable observation counter $count_{stable} = 0$.

- **Slow-Release (Optimize Latency)**: If the system is cleared of risk debt ($Debt_{risk} < \epsilon$, where ϵ is the debt clearance threshold) and has remained stable for a consecutive number of slots ($count_{stable} \geq N_{stable}$),

the controller cautiously decrements the advance time to reduce scheduling latency:

$$S_{next} = \max(S_{curr} - 1, 1) \quad (6)$$

and resets $count_{stable} = 0$.

- **Maintain Baseline**: Otherwise, the controller holds the current timing, $S_{next} = S_{curr}$, and increments $count_{stable}$ if $Risk_{pred} \leq 0$.

The core adjustment strategy is condensed in Algorithm 2.

Algorithm 2 Delay Management

Require: Δt_{arrive} (Observed timing offset), S_{curr} (Current trigger timing)

Ensure: S_{next} (Trigger timing for the next MAC schedule)

Notation:

μ_{delay} (EWMA mean delay), J_i (RFC 3550 Interarrival Jitter)

$Risk_{pred}$ (Predicted timing risk), $Debt_{risk}$ (History-weighted risk debt)

$count_{stable}$ (Consecutive stable arrivals counter)

N_{stable} (Adaptive stability threshold), S_{max} (Maximum advance-time limit)

α (Smoothing factor, where $\alpha = 1/8$)

ϵ (Debt clearance threshold)

Phase I: Risk and Debt Estimation

- 1: $error \leftarrow \Delta t_{arrive} - \mu_{delay}$
- 2: $\mu_{delay} \leftarrow \mu_{delay} + \alpha \cdot error$
- 3: $Risk_{pred} \leftarrow \max(\Delta t_{arrive}, \mu_{delay}) + J_i$
- 4: $Debt_{risk} \leftarrow Debt_{risk} + \alpha \cdot (\max(0, Risk_{pred}) - Debt_{risk})$
- 5: $IsDebtFree \leftarrow (Debt_{risk} < \epsilon)$

Phase II: Asymmetric Timing Adjustment (FASR)

- 6: **if** $\Delta t_{arrive} > 0 \vee Risk_{pred} > 0$ **then**
 - 7: $S_{next} \leftarrow \min(S_{curr} + 1, S_{max})$ ▷ Fast-Attack
 - 8: $count_{stable} \leftarrow 0$
 - 9: **else if** $IsDebtFree \wedge (count_{stable} \geq N_{stable})$ **then**
 - 10: $S_{next} \leftarrow \max(S_{curr} - 1, 1)$ ▷ Slow-Release
 - 11: $count_{stable} \leftarrow 0$
 - 12: **else**
 - 13: $S_{next} \leftarrow S_{curr}$ ▷ Maintain timing
 - 14: **if** $Risk_{pred} \leq 0$ **then**
 - 15: $count_{stable} \leftarrow count_{stable} + 1$
 - 16: **end if**
 - 17: **end if**
 - 18: **return** S_{next}
-

D. Coordination between Inner and Outer Loops

To ensure stable operation under real-world conditions, the inner and outer loops must coordinate without inducing control loop oscillations. The microsecond-level Node Sync (Inner-Loop) focuses strictly on correcting clock phase offsets to align the VNF's scheduling thread with the PNF's slot transitions. Because it directly shifts the VNF's time base, any clock adjustment ($S_{adj}, \mu s_{adj}$) temporarily shifts the arrival timestamp reference. The outer loop's Delay Manager (Outer-Loop) is designed to absorb this transient shifting. By relying

on the history-weighted risk debt ($Debt_{risk}$), the outer loop avoids reacting to the temporary timing offsets caused by clock adjustments, allowing the inner loop to converge. Once the inner loop locks synchronization ($sync_locked = 1$), the time base stabilizes, allowing the outer loop to achieve precise slot-level optimizations (S_{ahead}).

IV. EXPERIMENTAL RESULTS

Experiments were conducted to verify the proposed delay management method in an OpenAirInterface (OAI) nFAPI End-to-End environment. Each reported throughput value represents the average of 180 data points (3 independent trials $\times$ 60 samples per trial). We structure the evaluation into three parts: validating the EWMA mechanism (Section IV-C), benchmarking peak throughput and MAC Round-Trip Time (RTT) against static baselines (Section IV-D), and evaluating stability under network congestion (Section IV-E).

A. Experimental Scenario and rApp Framework

The testbed environment connects a series of high-performance servers, radio units, and a commercial smartphone. The detailed software and hardware configurations for the end-to-end network deployment are outlined below.

TABLE I
TESTBED NODE CONFIGURATION

Unit	Specification
Core Network (CN)	Open5GS
SW Stack / Branch	OpenAirInterface (2026.w13)
VNF Server	Intel® Xeon® Gold 6226R
PNF Server	Intel® Xeon® Gold 6433N
O-RU	Pegatron RU (P5G-Indoor O-RU-PR1450)
COTS UE	Samsung Galaxy S20

TABLE II
WIRELESS SYSTEM PARAMETERS

Parameter	Setting
Subcarrier Spacing	30 kHz
TDD Pattern	3D1S1U
MIMO / Ports	4-layer / 4T4R
Fronthaul	O-RAN F release

To ensure automated measurement and analysis with consistent data quality, an automated evaluation rApp was developed to operate on the Non-Real-Time RAN Intelligent Controller (Non-RT RIC) within the O-RAN Service Management and Orchestration (SMO) framework [14]. As shown in Fig. 2, this rApp automates the start-up of the VNF and PNF. Once the gNB is established, it automatically connects to the control PC via SSH to toggle the UE's airplane mode using Android Debug Bridge (ADB), and subsequently uses SSH to launch the iPerf server on the Core Network (CN) server. It then executes a series of pre-scheduled iPerf tests automatically. Finally, the rApp collects all log files via O1 SFTP and utilizes Python scripts to plot the following data analysis charts. Crucially, it manages the network latency simulation

at $T_5 - T_4$ and $T_2 - T_1$ via Linux Traffic Control (TC) to systematically validate the split-architecture stability.

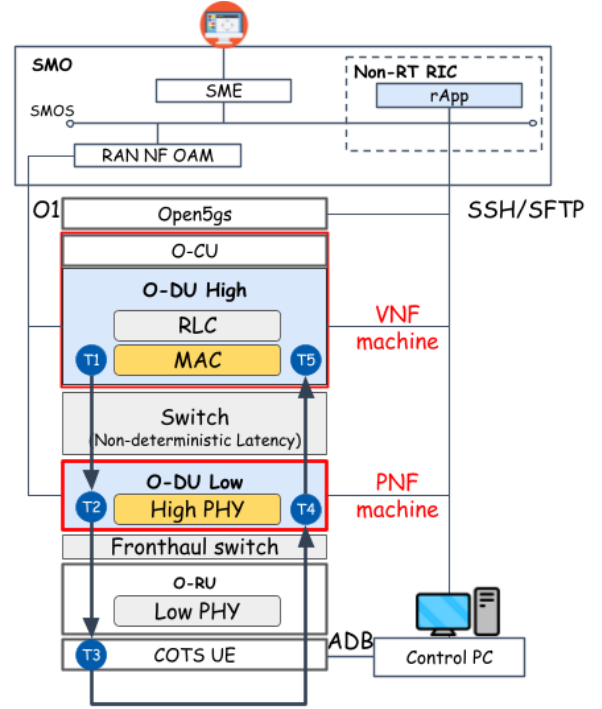


Fig. 2. Architecture of the rApp automated evaluation framework.

B. Rigorous System-Level Validation and Generalization

To prove the stability, generalization capability, and compatibility of the proposed dual-loop timing control and nFAPI modifications, our software implementation was submitted and successfully merged into the official upstream OpenAirInterface (OAI) repository. Beyond our localized end-to-end evaluation using the rApp framework, the proposed solution underwent rigorous testing in the official OAI Continuous Integration and Continuous Deployment (CI/CD) pipeline. Specifically, the code successfully passed all image building and cross-compilation tests across standard x86 and ARM64-based architectures, including enterprise-grade Red Hat Enterprise Linux (RHEL) clusters and NVIDIA Jetson edge platforms. Furthermore, our implementation successfully completed key system-level integration and physical layer regression tests. These tests include 5G Standalone (SA) end-to-end integration, real radio frequency (RF) transmissions using USRP B200 and AW2S radio units, standard F1/N2 interface control-plane handover procedures, and 3GPP-compliant physical layer channel simulations. These regression and compilation results confirm that our proposed software-defined timing loops are highly robust, backward-compatible, and ready for diverse commercial cloud hardware deployments without introducing any hardware dependency.

C. Scenario I: Validation of EWMA for PNF Timing Info

This scenario validates the application of an Exponentially Weighted Moving Average (EWMA) to smooth the PNF

Timing Info. Without EWMA processing, the raw timing information exhibits high variance due to network jitter, as shown in Fig. 3(a). Applying EWMA effectively filters out this high-frequency jitter (Fig. 3(b)), providing a stable baseline for the dynamic timing adjustment algorithm. Furthermore, as illustrated in Fig. 3(c) and Fig. 3(d), the smoothed data enables the system to reliably detect downward trends in the arrival offset (Δt_{arrive}). Therefore, when Δt_{arrive} begins to exhibit a downward trend, the algorithm can stably judge this condition and actively advance more ahead time to maintain synchronization.

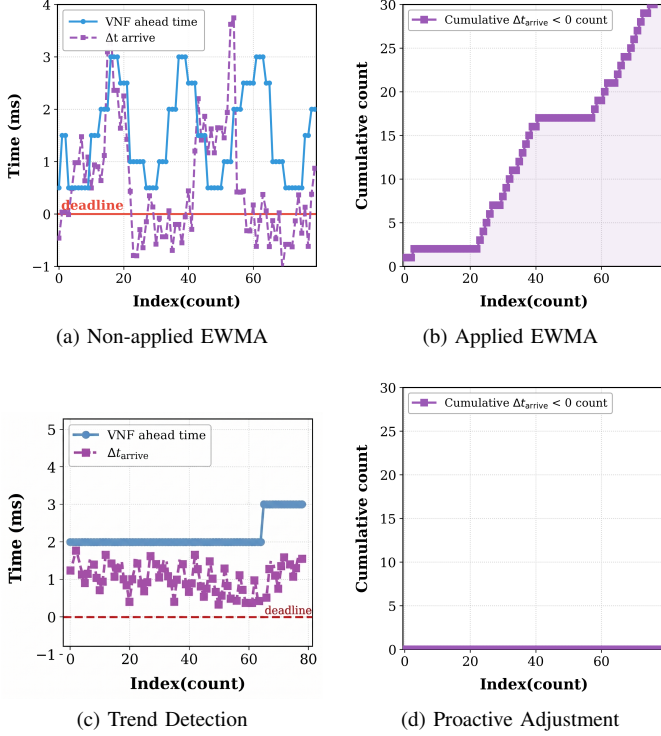


Fig. 3. Validation of EWMA processing on PNF Timing Info and proactive adjustment.

D. Scenario II: Performance Benchmark

We benchmark the peak throughput and MAC HARQ RTT against a static baseline. Under stable fronthaul conditions, as shown in Fig. 4, the proposed algorithm prioritizes guaranteeing the optimal throughput under different offered loads, consistently maintaining the best throughput performance. Simultaneously, while ensuring this optimal throughput, the algorithm also maximizes the selection of the best slot-ahead to optimize the MAC layer HARQ Round-Trip Time (RTT), effectively avoiding unnecessary overhead.

E. Scenario III: Stability under Network Congestion

To evaluate stability, we inject delay and jitter between the VNF and PNF using Linux TC. As shown in Fig. 5a, static slots ahead configurations suffer from synchronization failures and throughput drops when delay exceeds the timing

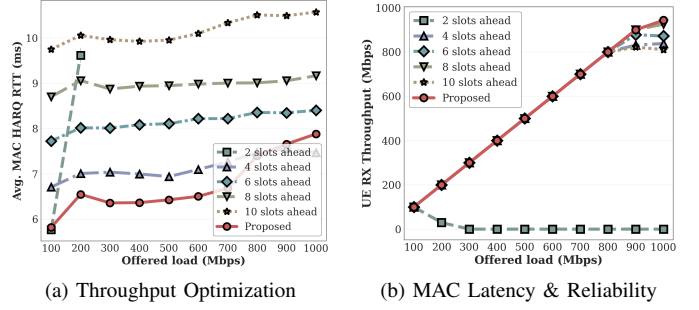


Fig. 4. Performance evaluation: MAC RTT efficiency and system throughput.

window, whereas our proposed adaptive mechanism dynamically increases the slots ahead to maintain stable throughput. Additionally, we evaluate the UE DL RX throughput under a saturated 1 Gbps UDP offered load with unpredictable link jitter. As shown in Fig. 5b, while fixed slot ahead configurations experience severe throughput degradation due to late packets, the proposed adaptive method dynamically tracks jitter and secures stable, high throughput at the UE.

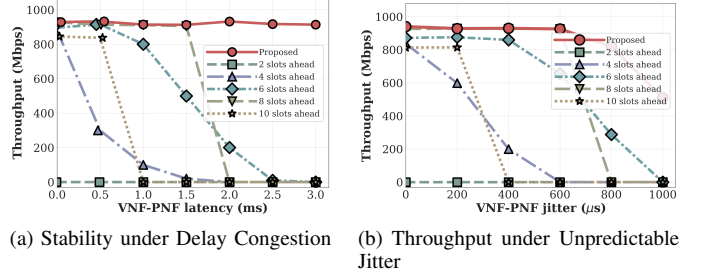


Fig. 5. System stability and throughput performance under varying network conditions.

V. CONCLUSION

This paper presented a dynamic timing adjustment framework for disaggregated RAN architectures, specifically focusing on the nFAPI-based Split Option 6 over non-ideal packet-switched networks. Recognizing the limitations of static slots ahead configurations under unpredictable network jitter and server processing delays, we proposed an Adaptive Slots Ahead mechanism based on Exponentially Weighted Moving Average (EWMA) to dynamically track and reduce network latency variations. Experimental validations conducted on the OpenAirInterface (OAI) platform with a commercial Pegatron O-RU demonstrated that the proposed method effectively eliminates synchronization failures and maintains stable system throughput, even under severe network congestion. Unlike existing solutions that rely on expensive hardware-level synchronization or specialized acceleration, our software-based approach offers a cost-effective and flexible alternative for deploying robust O-RAN systems on general-purpose Commercial Off-the-Shelf (COTS) servers.

REFERENCES

- [1] *TR 38.801 Study on new radio access technology: Radio access architecture and interfaces*, 3rd Generation Partnership Project (3GPP), 2017, version 14.0.0. [Online]. Available: <https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3066>
- [2] K. Alam, M. A. Habibi, M. Tammen, D. Krummacker, W. Saad, M. D. Renzo, T. Melodia, X. Costa-Pérez, M. Debbah, A. Dutta, and H. D. Schotten, “A comprehensive tutorial and survey of o-ran: Exploring slicing-aware architecture, deployment options, use cases, and challenges,” *IEEE Communications Surveys & Tutorials*, vol. 28, pp. 1637–1678, 2026.
- [3] O-RAN Alliance, “O-ran fronthaul working group control, user and synchronization plane specification,” O-RAN Alliance, Tech. Rep. O-RAN.WG4.CUS.0-v11.00, 2023.
- [4] Small Cell Forum (SCF), “5g network fapi (nfapi) specification,” Small Cell Forum (SCF), Tech. Rep. SCF225, 2020. [Online]. Available: <https://www.smallcellforum.org/docs/5g-nfapi-specifications/>
- [5] X. Foukas *et al.*, “Computational resource consumption of 5g virtualized ran,” in *2021 IEEE International Conference on Communications (ICC)*. IEEE, 2021.
- [6] D. Villa, I. Khan, F. Kaltenberger, N. Hedberg, R. S. da Silva, S. Maxenti, L. Bonati, A. Kelkar, C. Dick, E. Baena, J. M. Jornet, T. Melodia, M. Polese, and D. Koutsonikolas, “X5g: An open, programmable, multi-vendor, end-to-end, private 5g o-ran testbed with nvidia arc and openairinterface,” *IEEE Transactions on Mobile Computing*, vol. 24, no. 11, pp. 11 305–11 322, 2025.
- [7] S.-Q. Lee and M.-S. Lee, “A method of adaptive low-latency downlink transmission on fapi based 5g nr gnb,” in *2022 13th International Conference on Information and Communication Technology Convergence (ICTC)*. IEEE, 2022.
- [8] O. Adamuz-Hinojosa, L. Zanzi, V. Sciancalepore, A. Garcia-Saavedra, and X. Costa-Pérez, “Oranus: Latency-tailored orchestration via stochastic network calculus in 6g o-ran,” in *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*, 2024, pp. 61–70.
- [9] J. Lv, Y. Gao, Z. Ding, Y. Lin, X. You, G. Yang, and W. Dong, “Providing ue-level qos support by joint scheduling and orchestration for 5g vran,” in *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*, 2024, pp. 51–60.
- [10] V. G. Douros, A. Garcia-Saavedra, and C.-P. Xavier, “nfapi: The standard interface for sdn-based 5g small cell networks,” *IEEE Communications Standards Magazine*, vol. 2, no. 3, pp. 32–40, 2018.
- [11] X. Wang *et al.*, “An open-source implementation of the 5g nr functional split option 6,” in *2022 IEEE International Conference on Communications (ICC)*, 2022, pp. 1–6.
- [12] V. Paxson, M. Allman, J. Chu, and M. Sargent, “Computing tcp’s retransmission timer,” Internet Requests for Comments, RFC Editor, RFC 6298, June 2011. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc6298>
- [13] V. Jacobson, “Congestion avoidance and control,” *SIGCOMM Comput. Commun. Rev.*, vol. 18, no. 4, pp. 314–329, aug 1988. [Online]. Available: <https://doi.org/10.1145/52325.52356>
- [14] O-RAN Alliance, “O-ran architecture description,” O-RAN Alliance Working Group 1, Tech. Rep. O-RAN.WG1.O-RAN-Architecture-Description, 2024.